

계 획 서

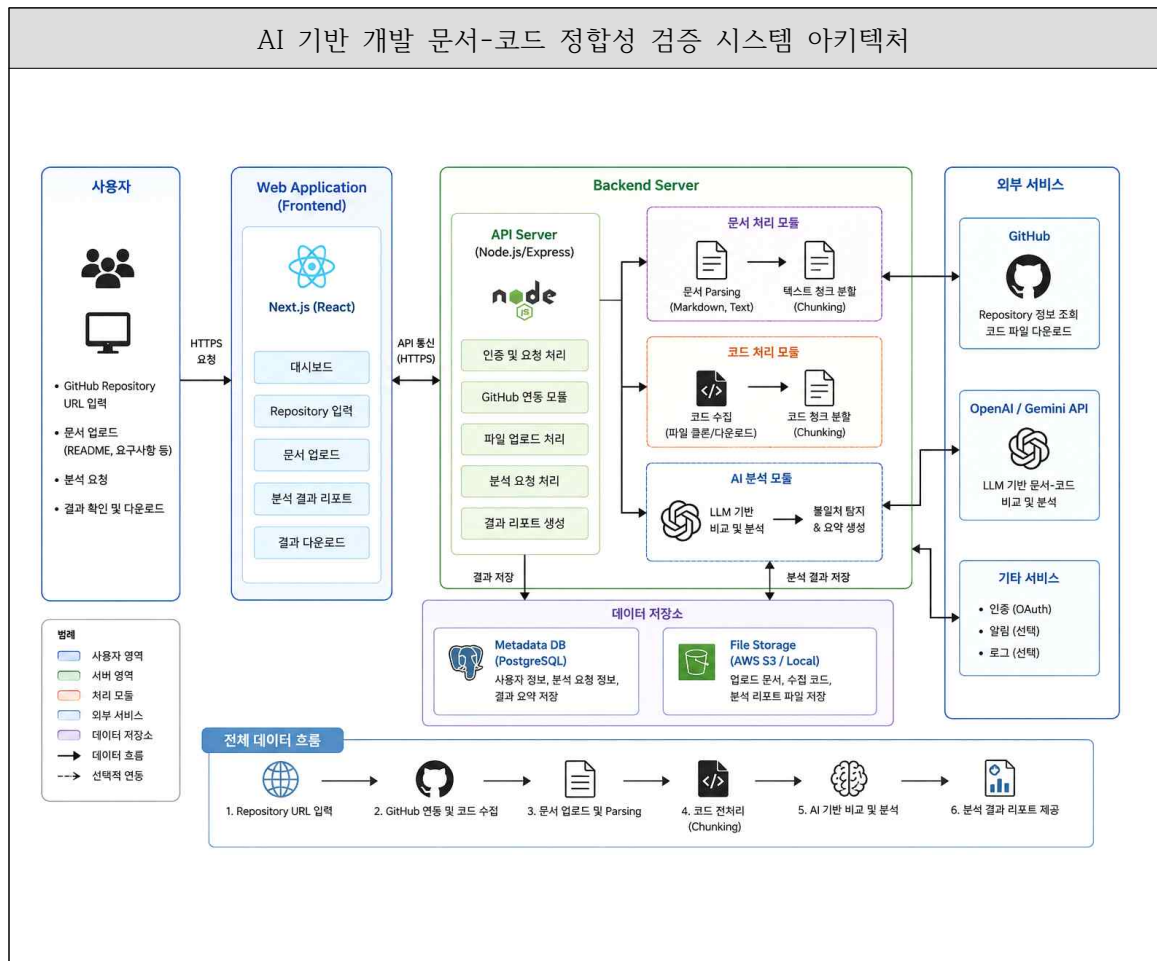
과제명 : AI 기반 개발 문서-코드 정합성 검증 시스템

팀명	오스터
팀장	이지수
팀원	황진영

1. 개발과제의 개요

본 과제는 소프트웨어 개발 과정에서 발생하는 개발 문서와 실제 구현 코드 간의 불일치를 AI 기반으로 자동 탐지하는 시스템을 개발하는 것을 목표로 한다. 실제 산업 현장에서는 요구사항 명세서, README, API 문서, 설계 문서 등 다양한 개발 문서가 지속적으로 수정되지만, 코드 변경 사항이 문서에 즉시 반영되지 않거나 반대로 문서에 명시된 기능이 코드에 구현되지 않는 문제가 빈번하게 발생한다. 이러한 문제는 유지보수 비용 증가, 협업 비효율, 신규 개발자의 프로젝트 이해도 저하 등의 원인으로 이어진다.

본 시스템은 사용자가 업로드한 개발 문서와 GitHub Repository 기반의 소스코드를 AI가 분석하여 기능 누락 가능성, API 불일치, 문서 outdated 가능성 등을 자동으로 탐지하고 리포트 형태로 제공한다. 이를 통해 기존 수작업 기반의 코드 리뷰 및 문서 검증 과정을 자동화하여 개발 생산성을 향상시키고, 유지보수 효율을 높이는 것을 목표로 한다.



또한 최근 산업 현장에서 활발히 활용되고 있는 생성형 AI 및 AI Agent 기술을 실제 DevOps Workflow에 접목하여, 단순 챗봇 수준을 넘어 실제 개발 프로세스 자동화에 활용 가능한 AI 서비스 모델을 제시하고자 한다.

기대효과는 다음과 같다.

- 개발 문서 검증 시간 단축
- 구현 누락 및 문서 불일치 조기 탐지
- 협업 과정에서의 커뮤니케이션 비용 감소
- 유지보수 효율 향상
- AI 기반 개발 자동화 workflow 경험 제공

활용방안은 다음과 같다.

- 기업 내부 개발 프로젝트 문서 검증
- GitHub 기반 Pull Request 리뷰 보조
- API 명세 및 실제 구현 비교
- 신규 개발자 온보딩 지원
- 유지보수 프로젝트의 문서 최신화 점검

2. 과제의 필요성 및 기대효과

(1) 기존 기술(과제)의 현황, 문제점 및 개선방안

최근 생성형 AI와 LLM(Large Language Model)의 발전으로 인해 개발 생산성을 향상시키기 위한 다양한 AI 기반 개발 도구들이 등장하고 있다. 대표적으로 GitHub Copilot, Cursor, Devin 등의 AI 기반 코드 보조 시스템이 개발 및 상용화되고 있으며, 기업 현장에서도 AI를 활용한 개발 자동화에 대한 관심이 증가하고 있다.

그러나 현재 대부분의 AI 개발 도구는 코드 생성, 코드 자동완성, 코드 리뷰 보조 등에 집중되어 있으며, 개발 문서와 실제 코드 간의 정합성을 지속적으로 검증하는 기능은 상대적으로 부족한 상황이다.

실제 산업 현장에서는 다음과 같은 문제가 빈번하게 발생한다.

- 문서에는 존재하지만 실제 코드에는 구현되지 않은 기능 존재
- 코드 수정 이후 문서 업데이트 누락
- API 명세와 실제 응답 형식 불일치

- 오래된 README 및 개발 가이드 유지
- 유지보수 과정에서 문서 신뢰도 저하

이러한 문제는 프로젝트 규모가 커질수록 더욱 심각해지며, 특히 협업 인원이 많은 프로젝트에서는 개발 생산성과 유지보수 효율을 크게 저하시킨다.

기존에는 이러한 문제를 코드 리뷰, QA, 테스트 코드, 정적 분석 도구 등을 통해 부분적으로 해결해왔지만, 자연어 기반 문서와 실제 구현 간의 의미적 차이를 자동으로 분석하는 데에는 한계가 존재한다.

본 과제에서는 생성형 AI 기반 semantic 분석 기법을 활용하여 개발 문서와 코드 간의 의미적 불일치를 탐지함으로써 기존 정적 분석 방식의 한계를 보완하고자 한다. 또한 단순 문서 검색 수준이 아닌, 실제 개발 workflow 내에서 활용 가능한 “AI 기반 개발 검증 Agent” 형태로 시스템을 설계하여 산업 현장에서 활용 가능한 방향으로 고도화하고자 한다.

(2) 과제개발 혹은 제작에 따른 기대효과

본 과제를 통해 기대되는 효과는 다음과 같다.

첫째, 개발 문서 검증 및 코드 리뷰 과정의 자동화를 통해 개발 생산성을 향상시킬 수 있다. 기존에는 개발자가 직접 문서와 코드를 비교해야 했으나, AI 기반 자동 분석을 통해 검토 시간을 단축할 수 있다.

둘째, 기능 누락 및 문서 불일치를 조기에 발견함으로써 유지보수 비용 감소 효과를 기대할 수 있다. 특히 협업 규모가 큰 프로젝트에서 문서 최신화 문제를 줄일 수 있다.

셋째, 신규 개발자의 프로젝트 적응 과정(Onboarding)을 지원할 수 있다. 최신 문서와 실제 코드 간의 차이를 자동으로 분석함으로써 프로젝트 이해도를 높일 수 있다.

넷째, 최근 산업 현장에서 중요성이 증가하고 있는 AI Agent 기반 개발 자동화 workflow를 직접 구현함으로써 실무형 AI 활용 사례를 제시할 수 있다.

다섯째, 향후 GitHub Actions, CI/CD Pipeline 등과 연동할 경우 실제 기업 환경에서도 활용 가능한 확장성을 확보할 수 있다.

3. 과제목표 및 내용

(1) 과제 수행 최종목표

본 과제의 최종목표는 개발 문서와 실제 코드 간의 주요 불일치 항목을 AI 기반으로 분석하여 사용자에게 리포트 형태로 제공하는 웹 기반 MVP(Minimum Viable Product) 시스템을 구현하는 것이다.

본 프로젝트는 단기간 내 구현 가능한 수준의 실용적인 AI Agent 시스템을 개발을 목표로 하며, 실제 산업 현장에서 발생하는 개발 문서 관리 및 코드 유지보수 문제를 해결하는 데 중점을 둔다.

1) 정량적 목표

항목	목표
GitHub 연동	Repository URL 입력 및 코드 수집 기능 구현
문서 업로드	README 및 요구사항 문서 업로드 기능 구현
AI 분석 기능	문서-코드 semantic 비교 기능 구현
불일치 탐지	최소 2종 이상의 불일치 유형 탐지
결과 출력	웹 기반 분석 리포트 제공
테스트	최소 3개 이상의 오픈소스 Repository 대상 테스트 수행

2) 정성적 목표

항목	목표
개발 생산성 향상	문서 검증 및 코드 리뷰 시간 단축
AI Agent 활용	개발 workflow 내 AI 활용 사례 제시
유지보수 효율 향상	문서 outdated 및 기능 누락 문제 감소
실무 적용 가능성	실제 개발 환경 적용 가능한 구조 설계

(2) 과제 내용

1) 주요 기능

기능	설명
개발 문서 업로드	README 및 요구사항 문서 업로드 지원
GitHub Repository 연동	GitHub URL 기반 코드 자동 수집
문서 Parsing	텍스트 기반 문서 Parsing 수행
코드 Chunking	코드 단위 분리 및 분석 전처리
AI 기반 비교 분석	문서와 코드 간 semantic 비교 수행
기능 누락 탐지	문서에 있으나 코드에 없는 기능 탐지
outdated 문서 탐지	코드 변경 대비 문서 최신화 여부 분석
결과 리포트 생성	분석 결과 및 권장 수정사항 출력
결과 시각화 UI	웹 기반 분석 결과 UI 제공

2) 비기능적 요구사항

구분	내용
성능	중소 규모 Repository 기준 안정적 분석 수행
응답속도	분석 요청 후 수 초 내 결과 반환 목표
유지보수성	Frontend / Backend / AI 모듈 분리 구조 설계
확장성	향후 PR 리뷰 및 CI/CD 연동 가능 구조 고려
사용성	최소 입력 기반의 직관적인 UI 제공
보안성	API Key 및 업로드 파일 세션 단위 관리

4. 추진전략 및 방법

(1) 추진전략 및 방법

본 과제는 단기간 내 구현 가능한 MVP 수준의 AI 기반 개발 검증 시스템 구축을 목표로 하며, 기존 생성형 AI API와 GitHub API를 적극 활용하여 개발 생산성을 높이는 방향으로 진행한다.

1) 개발 절차

단계	주요 내용
1단계	주제 구체화 및 계획서 작성
2단계	역할 분담 및 기존 기술 조사
3단계	Frontend / Backend 개발 환경 구축
4단계	GitHub API 연동 및 코드 수집 기능 구현
5단계	문서 업로드 및 Parsing 기능 구현
6단계	Prompt Engineering 기반 AI 분석 로직 구현
7단계	분석 결과 리포트 및 시각화 UI 구현
8단계	테스트 및 오류 수정
9단계	발표자료 제작 및 최종 정리

2) 적용 기술 및 방법론

구분	내용
Frontend	Next.js 기반 웹 UI
Backend	Node.js 기반 API 서버
AI 분석	OpenAI/Gemini API 기반 semantic 분석
코드 수집	GitHub API 활용
분석 방식	Prompt Engineering 기반 문서-코드 비교
협업 방식	GitHub 및 Notion 기반 협업

(2) 추진체계

역할	담당 내용
AI Workflow · 기능 구현 담당	문서-코드 비교 workflow 설계, Prompt Engineering, GitHub 연동 기능 구현, AI 분석 결과 개선 및 테스트
서비스 UI · 통합 담당	웹 UI 구현, 파일 업로드 및 결과 시각화 기능 구현, 사용자 흐름 개선, 시스템 통합 및 최종 배포

5. 추진일정

세부내용	수행기간(일)															비고
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	
주제 확정 및 계획서 작성	■															
역할 분담 및 기술 조사	■	■														
개발 환경 구축		■	■													
GitHub API 연동 구현			■	■												
문서 업로드 및 Parsing 구현			■	■												
AI 분석 로직 구현				■	■											
결과 리포트 UI 구현					■	■										
통합 테스트 및 오류 수정						■	■									
최종 배포 및 제출 자료 정리							■	■								

6. 참고문헌

- [1] Vaswani, A., Shazeer, N., Parmar, N., et al., "Attention Is All You Need," Advances in Neural Information Processing Systems (NeurIPS), 2017.
- [2] Lewis, P., Perez, E., Piktus, A., et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," Advances in Neural Information Processing Systems (NeurIPS), 2020.
- [3] Chen, M., Tworek, J., Jun, H., et al., "Evaluating Large Language Models Trained on Code," arXiv preprint arXiv:2107.03374, 2021.
- [4] GitHub Copilot 공식 문서 - AI 기반 코드 생성 및 개발 생산성 향상 사례 조사
- [5] Cursor AI Editor 공식 사이트 - AI 기반 코드 분석 및 개발 workflow 자동화 사례 조사
- [6] OpenAI API Documentation - LLM 기반 문서·코드 semantic 분석 및 Prompt Engineering 참고
- [7] GitHub REST API Documentation, GitHub Developers, 2025.
- [8] 미국 특허 US20230098156A1, "Systems and Methods for Automated Source Code Analysis Using Machine Learning," 2023.

* 구체적인 기술에 따른 아키텍처를 그림으로 붙여서 기술.